

Big Data & Data Engineering on Cloud

From On-Premise Roots to Cloud-Native Architectures

Le Ngoc Son – Inson@fit.hcmus.edu.vn



Lecture Agenda

12 Chapters • 2 Parts

PART 1 — Data Engineering: From On-Premise to Cloud

1



Foundations

Data Engineering role · SVs · data scale

2



The On-Premise Era

DW · ETL · Hadoop · Data Lake origins

3



Limitations of On-Premise

CAPEX · batch-only · ops burden · growth

4



Migration to Cloud

Why cloud · ETL→ELT shift · Batch vs Stream

5



Cloud Ecosystem

AWS / Azure / GCP services by layer

6

VN Use Cases — Vietnam

Tiki · Shopee · Grab · Zalo · MoMo · VPBank

7



Data Governance & Quality



6 dimensions · pipeline validation · DLP

8



Current Trends

Lakehouse · Streaming · AI/ML · Iceberg

PART 2 — BigQuery

BigQuery

9



BigQuery Introduction

Architecture · Dremel · Colossus · columnar

10



Working with BigQuery

SQL · ingestion · partition · security

11



Advanced BigQuery

ML in SQL · Lakehouse · pricing · vs Redshift

12



Summary & Q&A

Full recap · learning roadmap · discussion

CHAPTER 1

Foundations: Data & Data Engineering

Roles, lifecycle, Big Data 5Vs, and real-world data scale

1.1 What is Data Engineering?

Data Engineering is the practice of designing and building systems for collecting, storing, and processing data at scale — so analysts, scientists, and business teams can extract value from it reliably.



Data Engineer

Builds and maintains data pipelines, ETL/ELT processes, data warehouses, and lake infrastructure.



Data Analyst

Queries and visualizes curated data to answer specific business questions for decision-making.



Data Scientist

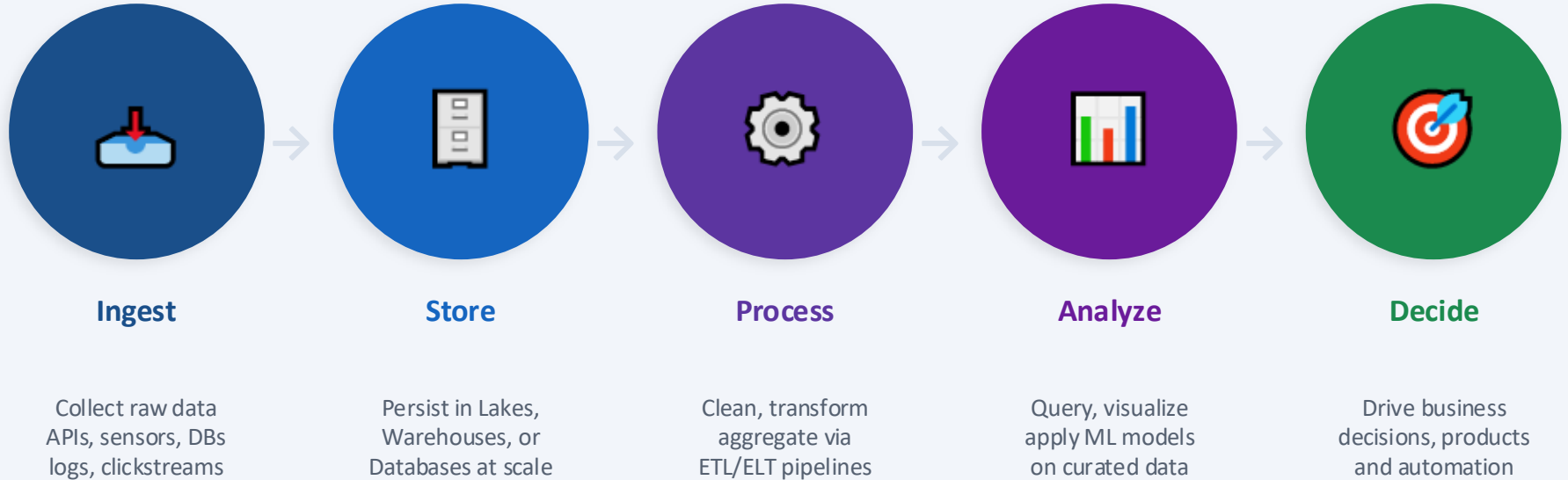
Applies ML and statistics to discover patterns and build predictive models on prepared data.



ML Engineer

Deploys and maintains ML models in production, bridging Data Science and Engineering.

1.2 The Data Lifecycle



Data Engineers own the first three stages — Ingest, Store, and Process — building the reliable data infrastructure that all downstream work depends on.

1.3 Big Data & the 5Vs Model



Volume

Scale of data
TB → PB → EB generated daily
worldwide



Velocity

Speed of data
Real-time streams & rapid
ingestion pipelines



Variety

Types of data
Structured, semi-structured,
unstructured



Veracity

Quality of data
Accuracy, completeness, trust
& reliability



Value

Business impact
Transforming raw data into
decisions & revenue



Shopee VN: millions of transactions/day (Volume), live inventory (Velocity), product images + JSON logs + reviews (Variety) → all 5Vs on one platform.

1.4 Data Scale in the Real World Today

2.5 QB

Data created daily worldwide

quintillion bytes

90%

of all data created in last 2 years

Exponential growth

175 ZB

Global datasphere by 2025

IDC estimate

5.3 B

Internet users generating data

as of 2024



Social Media

500M tweets/day; 95M
Instagram posts/day



E-Commerce

Amazon processes 66,000
orders/hour at peak



IoT & GPS

Grab: GPS ping every 2 seconds
per active driver



Finance

Visa handles 65,000
transactions/second globally

CHAPTER 2

The On-Premise Era: Architecture & Tools

Data Warehouses, ETL, Hadoop, and the rise of Data Lakes

2.1 Traditional Data Warehouse (DW)

A Data Warehouse is a centralized, structured repository optimized for analytical queries (OLAP) using a predefined schema. Data is loaded in batches via ETL.

Oracle Exadata

Microsoft SQL Server

Teradata

IBM Db2 Warehouse

Key traits: Schema-on-write • ACID transactions • SQL only • Structured data only

DW Architecture Layers



BI & Reporting Layer

Tableau, Power BI, Crystal Reports — end users query here



Data Marts

Department-specific subsets: Sales, Finance, HR



Core Data Warehouse

Integrated, historical, subject-oriented data store



ETL Staging Area

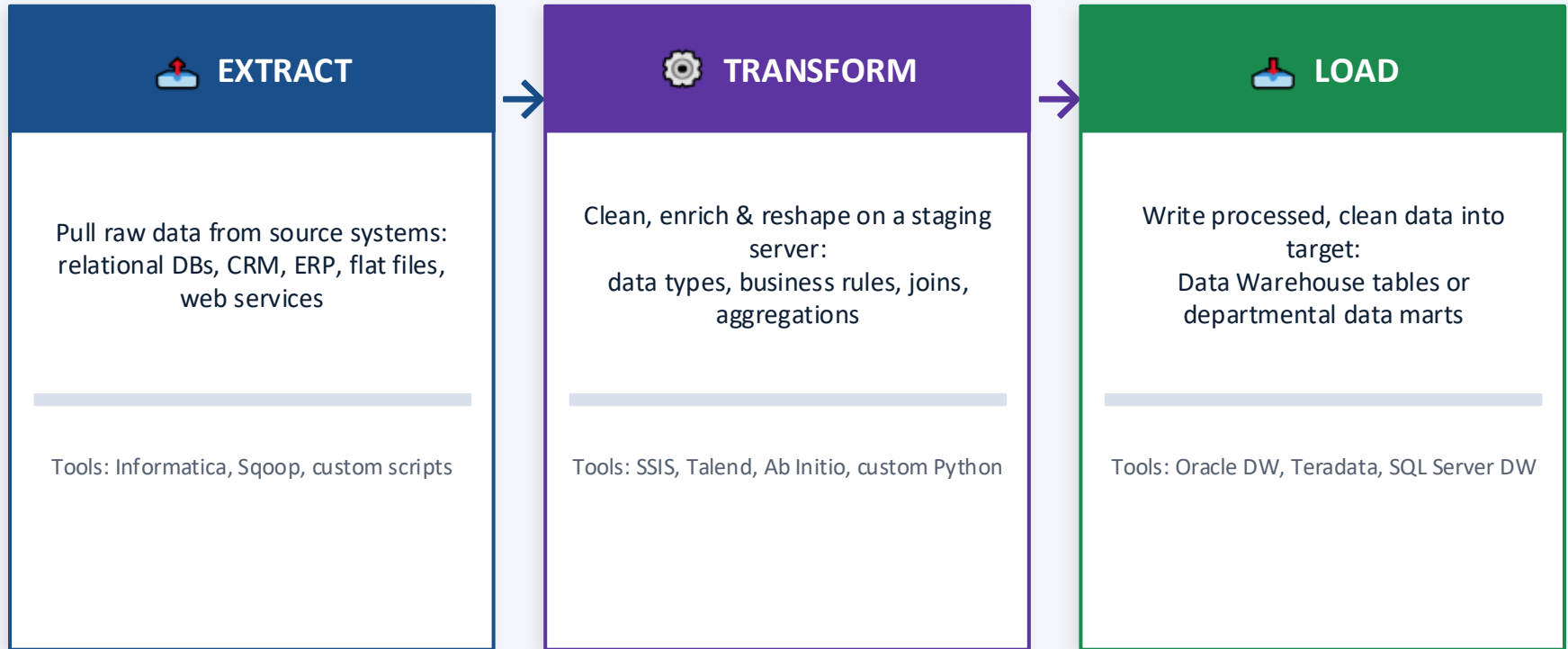
Extract → Transform → Load (runs nightly / weekly)



Source / OLTP Systems

Oracle, SQL Server, CRM, ERP operational databases

2.2 ETL — The Classic Data Processing Model



Core limitation: Transformation must happen BEFORE loading — requires a dedicated staging server and fails with unstructured/semi-structured data.

2.3 Hadoop Ecosystem — Distributed On-Premise Processing

HDFS

Hadoop Distributed File System. Stores data across commodity servers with 3x replication for fault tolerance. The 'hard drive' of the Hadoop cluster.

MapReduce

Parallel processing engine. Map tasks split work across nodes; Reduce tasks aggregate results. Batch-only with high latency (minutes to hours per job).

YARN

Yet Another Resource Negotiator. Schedules and manages compute resources across the cluster for different applications running simultaneously.

Apache Hive

SQL-on-Hadoop. Write HiveQL (SQL-like) queries that get translated into MapReduce jobs. Made Hadoop accessible to analysts who know SQL.

Apache Spark

In-memory processing engine. Up to 100x faster than MapReduce by keeping data in RAM. Supports batch, streaming, ML (MLlib), and SQL (Spark SQL).

Apache Pig

High-level scripting (Pig Latin) for ETL data pipelines on Hadoop. Less verbose than MapReduce Java code. Largely replaced by Spark today.

2.4 & 2.5 Data Lake — Origins & Comparison with Data Warehouse



Data Warehouse

Structure Schema-on-write (predefined)

Data Types Structured only (tables, rows)

Processing ETL — transform BEFORE load

Query Speed Fast (indexed, aggregated)

Flexibility Low — schema changes costly

Target Users BI analysts, executives

Cost High (enterprise licensing)

VS



Data Lake

Structure Schema-on-read (flexible)

Data Types Any: structured, JSON, images, logs

Processing ELT — transform AFTER load

Query Speed Slower (raw scanning)

Flexibility High — store now, define later

Target Users Data scientists, DE teams

Cost Low (commodity object storage)

CHAPTER 3

Limitations of On-Premise Systems

Infrastructure, performance, operations, and growth
bottlenecks

3.1 – 3.4 The Four Major Limitations of On-Premise



3.1 Infrastructure & Cost

Massive CAPEX: servers, storage arrays, networking, data center space and cooling. Scale-up means buying bigger hardware. Idle capacity wastes money year-round. Typical enterprise DW project: \$2M–\$10M upfront before writing a single query.



3.2 Performance & Real-Time Gaps

ETL batch jobs run overnight — analysts see yesterday's data, never today's. Real-time use cases (fraud detection in < 1s, surge pricing, live dashboards) are architecturally impossible. MapReduce jobs take minutes to hours, not milliseconds.



3.3 Operational Complexity

Requires permanent teams of DBAs, sysadmins, and network engineers. Patching, backups, and disaster recovery planning consume 40–60% of IT budgets. Any version upgrade or schema change risks downtime and data corruption.



3.4 Can't Keep Up with Data Growth

Data volume doubles roughly every two years. Unstructured data (JSON, images, audio, sensor streams) simply doesn't fit relational schemas. IoT devices and mobile apps generate events faster than on-premise pipelines can absorb them.

CHAPTER 4

Migration to Cloud & ETL → ELT Shift

Why cloud, the key paradigm change, batch vs stream

4.1 Why Cloud? Core Value Propositions



Elastic Scaling

Scale to thousands of nodes in minutes; back to zero when idle. Grab scales its Spark cluster 10x during peak hours, then releases resources automatically.



Pay-as-you-go

CAPEX → OPEX. Pay only for what you use. No upfront hardware. A startup can run enterprise-grade analytics for a few hundred dollars per month.



Fully Managed Services

Cloud provider handles patching, high-availability, backups, and DR. Your team focuses on data pipelines and business logic — not server administration.



Global Infrastructure

30+ regions worldwide. Low-latency access globally. Multi-region disaster recovery is a built-in feature, not a multi-million-dollar separate project.



Ecosystem Integration

Storage + Compute + ML + BI + Streaming pre-integrated. GCS → Dataflow → BigQuery → Looker Studio is a coherent, single-vendor stack.



Enterprise Security

ISO 27001, SOC 2, GDPR compliance out of the box. IAM, VPC, encryption at rest/transit. Often more secure than most on-premise deployments.

4.2 On-Premise vs Cloud — Direct Comparison

Category	On-Premise	Cloud
Provisioning	Weeks to months (procurement cycle)	Minutes via console or API
Cost Model	CAPEX — large upfront investment	OPEX — pay per use, no minimum
Scaling	Manual, slow, capacity limits	Auto-scale, elastic, unlimited
Maintenance	Full team: DBA, network, sysadmin	Managed — provider handles all
Disaster Recovery	Complex, expensive separate DR site	Built-in multi-region replication
Innovation Speed	Months for new versions/tools	Instant access to latest services
Security	Self-managed (variable quality)	Enterprise-grade, certified compliance
Cost Efficiency	Pay for idle capacity 24/7	Pay only when workloads run

4.3 ETL → ELT: The Most Important Mindset Shift (1/2 — The Why)



Key question: Why did "Transform before Load" (ETL) dominate for 30 years — and why does Cloud make "Load then Transform" (ELT) superior?



Why ETL was necessary On-Premise

✗ Storage was expensive

On-premise disk cost \$1,000s per TB. Loading raw, dirty data was wasteful. Transform first, load only clean records.

✗ DW compute was finite

The warehouse server had fixed CPU/RAM. Pre-aggregate everything outside to reduce the load on the expensive DW.

✗ No elastic compute

Staging server did the heavy lifting. If transformation failed, you couldn't just spin up 100 more servers.



Why ELT works on Cloud

✓ Storage is nearly free

GCS / S3 costs ~\$0.02/GB/month. Load everything raw — it's cheap. Worry about structure only when you query.

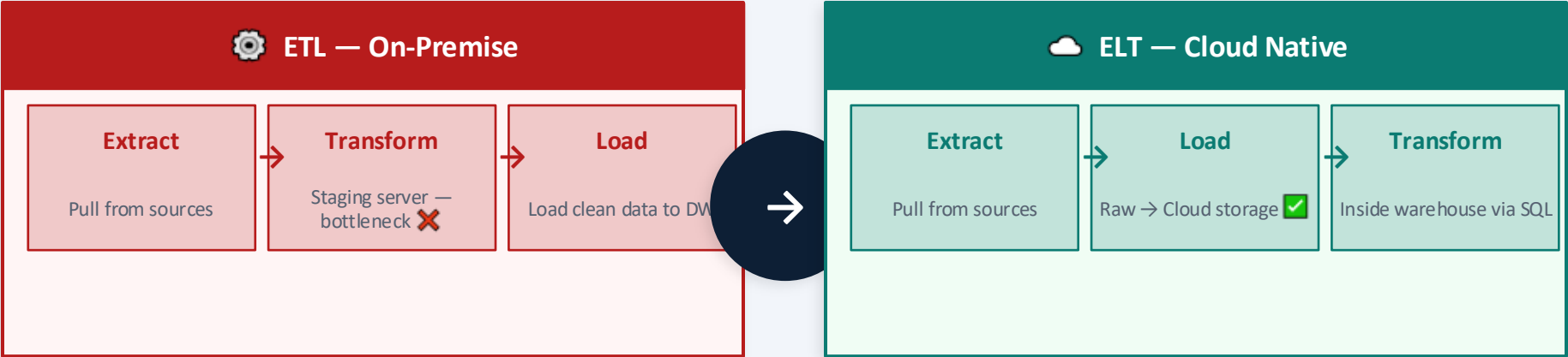
✓ Infinite compute on demand

BigQuery / Redshift scale to thousands of workers per query. The 'Transform' step runs INSIDE the warehouse at petabyte scale.

✓ SQL is the transformation

dbt, Dataform, Spark SQL — write SELECT statements and let the cloud engine do the rest. No staging server needed at all.

4.3 ETL → ELT: Side-by-Side & Modern Cloud Tools (2/2)



	ETL	ELT (Cloud)
Transform at	Staging server (fixed)	Inside cloud warehouse (elastic)
Data types	Structured only	Any — raw JSON, logs, images OK
Flexibility	Schema must be defined up-front	Schema defined when querying
Tools	SSIS, Informatica, Talend	dbt, Dataform, Spark SQL
Cost driver	Staging server hardware	Cloud compute (per query)

4.4 Batch Processing vs Stream Processing on Cloud



Batch Processing

When	Scheduled: hourly, nightly, weekly
Latency	Minutes to hours
Data Model	Bounded datasets — finite size
Use Cases	Payroll, monthly reports, DW refresh, ML retraining
Cloud Tools	AWS Glue, Azure Data Factory, GCP Dataproc
Strength	High throughput; simpler to build and debug
Weakness	Stale data — not suitable for real-time decisions



Stream Processing

When	Continuous — data processed as it arrives
Latency	Milliseconds to seconds
Data Model	Unbounded streams — infinite, ongoing
Use Cases	Fraud detection, surge pricing, live dashboards, IoT
Cloud Tools	Kinesis, Azure Event Hub, GCP Pub/Sub + Dataflow
Strength	Real-time decisions; immediate insight from fresh data
Weakness	Complex to build; stateful operations are tricky

CHAPTER 5

Cloud Data Engineering Ecosystem

AWS, Azure & GCP — services organized by architectural layer

5.1 Data Engineering Service Layers: Ingest → Store → Process → Analyze

 INGEST	Collect data from all sources into cloud	AWS Kinesis, DMS, Snowball	Azure Event Hub, Data Box	GCP Pub/Sub, Transfer Svc
 STORE	Persist raw and curated data at any scale	AWS S3, RDS, DynamoDB	Azure ADLS Gen2, Cosmos	GCP GCS, Bigtable, Spanner
 PROCESS	Transform, aggregate, orchestrate pipelines	AWS Glue, EMR, Lambda	Azure Data Factory, HDInsight	GCP Dataflow, Dataproc, Composer
 ANALYZE	Query, visualize, derive business insights	AWS Redshift, Athena	Azure Synapse, Power BI	GCP BigQuery, Looker Studio

5.2 – 5.4 AWS / Azure / GCP — Key Services Deep Dive

AWS

S3 — Object Storage / Data Lake

Kinesis — Real-time streaming

Glue — Serverless ETL + Data Catalog

EMR — Managed Spark / Hadoop cluster

Redshift — Cloud Data Warehouse

Athena — Serverless SQL query on S3

Azure

ADLS Gen2 — Data Lake Storage

Event Hub — Streaming ingestion

Data Factory — ETL / ELT pipelines

HDInsight — Managed Spark / Hadoop

Synapse Analytics — DW + Spark + SQL

Power BI — BI & visualization

GCP

GCS — Object Storage / Data Lake

Pub/Sub — Messaging & streaming

Dataflow — Apache Beam ETL / streaming

Dataproc — Managed Spark / Hadoop

BigQuery — Serverless DW ★

Looker Studio — BI & dashboards

5.5 Equivalent Services Across AWS / Azure / GCP

Category	AWS	Azure	GCP
Object Storage	S3	Blob / ADLS Gen2	Cloud Storage (GCS)
Stream Ingestion	Kinesis	Event Hub	Pub/Sub
ETL / ELT	Glue	Data Factory	Dataflow / Dataform
Managed Spark	EMR	HDInsight	Dataproc
Data Warehouse	Redshift	Synapse Analytics	BigQuery 🌟
Serverless Query	Athena	Synapse Serverless SQL	BigQuery (default mode)
Orchestration	MWAA (Airflow) / Step Fn	Data Factory Pipelines	Cloud Composer (Airflow)
BI & Visualization	QuickSight	Power BI	Looker Studio
ML Platform	SageMaker	Azure Machine Learning	Vertex AI

CHAPTER 6

Real-World Use Cases in Vietnam

Cloud Data Engineering applied at scale in Vietnamese companies

6.1 & 6.2 E-Commerce & Ride-Hailing: Tiki, Shopee, Grab



Tiki

Challenge: Personalized recommendations for 20M+ users; real-time inventory updates during flash sales

Stack: Kinesis → EMR / Spark → Redshift (DW) → QuickSight (BI) + SageMaker (ML recommendations)

AWS

✓ 60% reduction in pipeline processing time. Recommendation models updated hourly from fresh clickstream data.



Shopee VN

Challenge: 11.11 flash sale: 50x traffic spike. Process 5M+ orders in 24 hours — zero downtime allowed

Stack: Auto-scaling EMR → S3 Data Lake → Redshift → SageMaker fraud scoring → real-time seller dashboard

AWS

✓ 99.99% uptime during peak. Seller analytics dashboard refreshed every 30 seconds automatically.



Grab VN

Challenge: Track 50,000+ drivers in real-time, compute surge pricing dynamically, optimize route assignments

Stack: Pub/Sub (GPS ping every 2s) → Dataflow (stream processing) → BigQuery (analytics) → Vertex AI (ML)

GCP

✓ Surge pricing computed in < 200ms. Over 100TB GPS data analyzed monthly inside BigQuery.

6.3 & 6.4 Super App & Fintech: Zalo, MoMo, VPBank



Zalo

Hybrid

Challenge: 800M+ messages/day. Real-time A/B testing on features. User retention analytics for 74M users

Stack: Kafka (ingest) → Flink (stream) → GCS (lake) → BigQuery (analytics) → custom ML inference pipeline

✅ A/B test results available in < 5 minutes. Message delivery rate optimized to 99.97% uptime.



MoMo

AWS + GCP

Challenge: 30M+ digital wallet users. Real-time transaction monitoring and daily GMV reporting for C-suite

Stack: Kinesis → Glue ETL → S3 → Redshift → QuickSight + BigQuery ML for fraud probability scoring

✅ Fraud detection latency reduced from 2 hours to 800ms. Executive dashboards automated daily.



VPBank

Azure

Challenge: Core banking data + customer behavior analytics. Regulatory reporting. Credit risk scoring pipeline

Stack: Event Hub (transactions) → Data Factory (ETL) → Synapse Analytics (DW) → Power BI + Azure ML

✅ Credit risk model accuracy improved 23%. Regulatory reports automated from 3 days to 2 hours.

6.5 Common Patterns — What All These Companies Share

- ① Event-driven ingestion**

All companies use streaming ingestion (Kafka / Pub/Sub / Kinesis) to capture events in real-time — not file uploads or overnight batch pulls from databases.
- ② Decoupled storage & compute**

Object storage (S3 / GCS / ADLS) acts as the source of truth. Compute (Spark / Dataflow / BigQuery) is spun up on demand and released after each job.
- ③ Lambda or Kappa architecture**

Parallel batch and stream pipelines feed a serving layer (DW or cache). This ensures both historical accuracy and real-time freshness for the same dataset.
- ④ SQL + ML integration**

Teams use SQL for 80% of analytics and ML for specialized tasks (fraud, recommendation). BigQuery ML, SageMaker, and Vertex AI make this seamless.
- ⑤ Observability & data quality**

Production pipelines include automated quality checks, pipeline monitoring with SLA alerting, and data lineage tracking. Bad data is caught before dashboards.

Data Governance & Data Quality

"80% of a Data Engineer's time is spent cleaning bad data"

7.1 What is Data Governance?

Data Governance is the set of policies, standards, processes, and roles that ensure data across an organization is accurate, consistent, secure, and used appropriately throughout its lifecycle.



Data Catalog

Centralized inventory of all data assets. Documents where data lives, what it means, who owns it, and how it is used. Tools: Google Data Catalog, AWS Glue Catalog, Apache Atlas.



Data Lineage

Tracks data from origin to consumption: which pipeline produced this column? Which source table? Lineage makes debugging, auditing, and impact analysis possible.



Metadata Management

Business and technical metadata attached to every dataset: description, owner, update frequency, PII sensitivity, quality score. Enables discovery and trust.



Data Stewardship

Assigning human accountability. Data Owners define policies; Data Stewards enforce quality; Data Consumers follow access and usage rules across all teams.

7.2 The Data Quality Problem — Why Bad Data is Expensive



Gartner estimates poor data quality costs organizations an average of \$12.9 million per year. Bad data → wrong decisions → failed ML models → legal risk.

The 6 Dimensions of Data Quality

Accuracy

Data correctly reflects the real-world entity it represents.

e.g. A user's phone number stored as '0987654321' must match their actual number

Completeness

All required fields are present; no critical values are missing.

e.g. An order record with NULL customer_id is incomplete — cannot fulfill

Consistency

Same data across systems agrees; no contradictions between tables.

e.g. User age = 25 in CRM but 52 in analytics DW — which is correct?

Timeliness

Data is available when needed and reflects recent reality.

e.g. Inventory data 6 hours old during a flash sale causes overselling

Uniqueness

No duplicates exist that would distort aggregations and counts.

e.g. Same customer counted twice in revenue report inflates GMV by 2x

Validity

Data conforms to defined formats, ranges, and business rules.

e.g. Order date '2024-13-45' or negative price are invalid records

7.3 Data Quality in Pipelines — Validate, Quarantine, Fix

Key principle: "Shift quality left" — catch bad data as early in the pipeline as possible, before it reaches dashboards, reports, or ML training sets.



At Ingest

- ✓ Schema validation: correct column types?
- ✓ Null checks: are required fields populated?
- ✓ Range checks: is price > 0? is age between 0–120?
- ✓ Referential integrity: does customer_id exist?



In Transform

- ✓ Deduplication: remove exact & fuzzy duplicates
- ✓ Standardization: phone/date/address formats
- ✓ Business rule checks: qty * price = total?
- ✓ Anomaly detection: 10x spike in orders normal?



At Serving

- ✓ Row count assertion: did pipeline produce expected rows?
- ✓ Freshness check: was table updated in last 24h?
- ✓ Distribution drift: has value distribution shifted?
- ✓ Cross-table consistency: do aggregations reconcile?

Tools: Great Expectations • Soda Core • dbt tests • Apache Griffin • AWS Deequ • Dataform assertions

7.4 Data Governance on Cloud — Controls, Privacy & Scale



Row & Column Level Security

BigQuery, Synapse, and Redshift support row-level security (different users see different rows from same table) and column-level policies (mask or hide PII columns like SSN, salary, email from unauthorized roles).



PII Detection & Masking

GCP DLP API, AWS Macie, and Azure Purview automatically scan datasets to detect sensitive data (names, phone numbers, credit cards). Detected PII can be masked, tokenized, or redacted in the pipeline.



Audit Trails & Compliance

All queries, data access, and schema changes are logged automatically. Enables GDPR data lineage requests, SOC 2 audit evidence, and regulatory compliance reporting without manual effort.



Data Mesh as Governance Model

Decentralize data ownership to domain teams — each team owns its data products with defined SLAs and quality contracts. Central platform team enforces shared standards (schema registry, PII policies, access control) across all domains.

CHAPTER 8

Current Trends in Cloud Data Engineering

Lakehouse, streaming-first, AI/ML pipelines, open formats, serverless

8.1 Data Lakehouse — Unifying Lake + Warehouse

The Lakehouse combines the low-cost, flexible storage of a Data Lake with the ACID transactions, SQL performance, and governance of a Data Warehouse — in a single unified architecture.

Data Lake	Data Warehouse	Lakehouse
✓ Low-cost storage ✓ Any data type ✓ Schema flexible ✓ ML-friendly ✗ No ACID transactions ✗ Poor SQL performance ✗ Hard to govern	✓ Fast SQL queries ✓ ACID + strong types ✓ Strong governance ✓ BI-ready ✗ Expensive at scale ✗ Structured only ✗ Rigid schema	✓ Low-cost storage ✓ ACID transactions ✓ SQL + ML + Streaming ✓ Open formats 🏆 Best of both worlds

8.2 & 8.3 Modern ELT Tools & Streaming-First Architecture

8.2 Modern ELT Tools (dbt & Dataform)

dbt (data build tool)

Write SQL SELECT statements; dbt handles CREATE TABLE / VIEW. Version-controlled SQL transformations inside BigQuery or Redshift. Used by 40,000+ companies globally.

Google Dataform

GCP-native ELT tool. Define transformation pipelines as SQL workflows with scheduling, dependency management, and data quality assertions — all inside GCP.

Apache Airflow

Open-source workflow orchestrator. Define DAGs (Directed Acyclic Graphs) of tasks. Schedule and monitor your entire ELT pipeline. Managed via Cloud Composer on GCP.

8.3 Streaming-First Architecture



Why streaming-first?

Businesses need decisions NOW — not tomorrow. Fraud caught after 1s costs 10x more than fraud caught in 100ms. Streaming is the new default.



Kappa Architecture

Single streaming pipeline replaces separate batch + stream stacks. All history is replayable from the event log (Kafka / Pub/Sub). Simpler ops.



Apache Flink

Stateful stream processor. Sub-second latency. Used by Alibaba to process trillions of events on 11.11. Supports complex window operations.



GCP Dataflow

Managed Apache Beam. Unified batch + streaming in one API. Auto-scales workers per query. No cluster to manage — fully serverless.

8.4 – 8.6 AI/ML in Pipelines, Open Table Formats & Serverless

8.4 AI/ML Embedded in Pipelines

Feature Stores

Centralized ML feature repository (Feast, Vertex AI Feature Store). Ensures training-serving consistency — no skew between what model learned and what it sees in production.

MLOps Pipelines

CI/CD for ML: automated training, evaluation, deployment triggered by data drift or new data. Vertex AI Pipelines, SageMaker Pipelines.

BigQuery ML

Train and serve ML models using pure SQL inside BigQuery — no Python required. Linear, logistic, k-means, ARIMA, DNN all supported. (Covered in Part 2.)

8.5 Open Table Formats

Apache Iceberg

ACID transactions on Data Lakes. Time-travel queries. Snapshot isolation. Supported natively in BigQuery, Spark, Flink, and Athena — the most widely adopted format.

Delta Lake

Open-source from Databricks. ACID tables on Parquet files. DML operations (UPDATE / DELETE / MERGE) on the lake. Foundation of the Databricks Lakehouse platform.

Apache Hudi

Record-level upserts and deletes on Data Lakes. Near-real-time ingestion. Used heavily at Uber and ByteDance for streaming upsert workloads.

8.6 Serverless & Zero-Ops

BigQuery

Zero cluster management. Run SQL on petabyte-scale data. Pay per TB scanned. Infrastructure completely invisible to users.

AWS Athena

Query S3 directly with ANSI SQL. No servers, no cluster setup. Pay \$5 per TB scanned. Instant results on data lake files.

GCP Dataflow

Auto-scaled Apache Beam pipelines. No Spark cluster to provision or tune. Google manages all worker lifecycle automatically.

PART 2

Part 2: BigQuery — Google Cloud Data Warehouse

Architecture · SQL · ML · Lakehouse · Pricing

CHAPTER 9

Introduction to BigQuery

What it is, how it fits in GCP, and how it works inside

9.1 What is BigQuery?

BigQuery is Google Cloud's fully managed, serverless data warehouse that enables scalable analysis over petabytes of data using familiar SQL — with zero infrastructure to manage.

2010

Year launched

< 10 sec

TB-scale query time

99.99%

Uptime SLA

50,000+

Global customers

BigQuery



Fully Managed



Serverless



Standard SQL (ANSI 2011)



Petabyte-scale queries



Built-in ML (no Python!)



Multi-cloud (Omni)



Enterprise Security



Pay per query / slot

9.2 BigQuery Architecture — Separation of Storage & Compute



Game-changing insight: Storage and Compute are fully decoupled — each scales independently and is billed separately. This is the foundation of BigQuery's power.

Dremel

Distributed Query Engine

Breaks SQL into a tree of sub-tasks. Thousands of workers execute in parallel across column slices. A 1TB query fans out to 3,000+ workers in milliseconds — that's why BigQuery is fast.

Colossus

Distributed File System

Google's distributed storage system. Data stored in columnar format (Capacitor files). Replicated across multiple zones automatically. Completely decoupled from compute — storage never 'runs out.'

Borg / Kubernetes

Compute Orchestrator

Google's container management system. Allocates compute resources from a shared pool across thousands of workers for each query. Resources are released immediately after the query completes.

Jupiter Network

Internal Network Fabric

Google's internal 1 Petabit/second network connecting Colossus (storage) to Dremel (compute). This massive bandwidth makes reading data from storage nearly as fast as reading from local disk.

9.3 Columnar Storage — Why BigQuery Queries Are So Fast

✗ Row-Based Storage (OLTP)

ID	Name	Amount	Region
1	Alice	\$500	HCM
2	Bob	\$320	HN
3	Carol	\$980	DN

Query: `SELECT SUM(Amount)`
Reads ALL 4 columns per row!
ID, Name, Region scanned too.
Wasted I/O = slow performance.



✓ Columnar Storage (BigQuery)

ID	Name	Amount	Region
1	Alice	500	HCM
2	Bob	320	HN
3	Carol	980	DN

Query: `SELECT SUM(Amount)`
Reads ONLY the Amount column!
ID, Name, Region: never touched.
10–100x less I/O = much faster!

CHAPTER 10

Working with BigQuery

Data ingestion, SQL capabilities, optimization, and security

10.1 Loading Data into BigQuery — Ingestion Methods

① Batch Load

Upload CSV, JSON, Avro, Parquet, ORC from Cloud Storage or local files. Scheduled or on-demand. Loading itself is FREE — you only pay for storage afterward.

② Streaming Insert

insertAll API: push rows in real-time (< 1s latency). Data is queryable immediately after insertion. Ideal for live event tracking and IoT streams. Charged per row.

③ Storage Write API

High-throughput streaming (1M+ rows/sec). Supports exactly-once semantics for production reliability. Google recommends this over legacy insertAll for new pipelines.

④ Federated Query

Query CSV, JSON, or Parquet files on GCS directly with SQL — without loading data into BigQuery. No ETL needed. Great for ad-hoc exploration of raw lake data.

⑤ Data Transfer Service

Scheduled connectors that auto-load data from Google Ads, Analytics 360, YouTube, Amazon S3, and Teradata into BigQuery on a configurable schedule.

10.2 Writing SQL in BigQuery — Key Capabilities

Top 5 Sellers by Region — Window Function Example

```
SELECT
  seller_id,
  seller_name,
  SUM(order_amount) AS revenue,
  RANK() OVER (
    PARTITION BY region
    ORDER BY SUM(order_amount) DESC
  ) AS rank_in_region
FROM `shopee_vn.orders`
WHERE DATE(created_at)
  BETWEEN '2024-01-01'
  AND '2024-12-31'
GROUP BY 1, 2
QUALIFY rank_in_region <= 5;
```

Key SQL Capabilities in BigQuery

◆ Standard SQL (ANSI 2011)

Full SQL: subqueries, CTEs (WITH clause), set operations

◆ Window Functions

RANK, DENSE_RANK, ROW_NUMBER, LEAD, LAG, NTILE, PERCENT_RANK

◆ ARRAY & STRUCT types

Nested and repeated fields — native JSON-like data structures

◆ QUALIFY clause

Filter on window function results (unique to BigQuery dialect)

◆ User-Defined Functions

JavaScript or SQL UDFs for custom transformation logic

◆ INFORMATION_SCHEMA

Query metadata: table sizes, slot usage, query history, jobs

10.3 & 10.4 Optimization (Partition, Cluster) & Security

10.3 Cost & Performance Optimization

Partitioning

Divide table by date/range/ingestion time. Queries with date filters scan only relevant partitions → up to 90% cost reduction. Defined at CREATE TABLE time.

Clustering

Sort data within partitions by column values. Filters on cluster columns (seller_id, region, category) scan far fewer storage blocks — faster and cheaper.

Materialized Views

Pre-computed query results stored as a table. BigQuery auto-refreshes when base data changes. Dramatically speeds up repeated dashboard and reporting queries.

10.4 Security & Governance

IAM Roles

Grant access at project, dataset, or table level. Viewer / Editor / Admin / Custom roles with fine-grained permissions.

Column-level Security

Policy tags restrict sensitive columns (PII, salary, SSN) from unauthorized roles — without creating separate tables.

Row-level Security

Filter what rows a user sees based on their identity. Same table, different result sets per user group or region.

VPC Service Controls

Isolate BigQuery within a network perimeter. Prevents data exfiltration even by authorized users.

CMEK Encryption

Customer-managed encryption keys via Cloud KMS. You control when and how data can be decrypted.

Cloud Audit Logs

Every query, access event, and schema change logged automatically — full GDPR and SOC 2 compliance trail.

CHAPTER 11

Advanced BigQuery

ML in SQL, Lakehouse integration, pricing & competitive comparison

11.1 BigQuery ML — Train Machine Learning Models with SQL

Customer Churn Prediction — Full Workflow in SQL

```
-- Step 1: Train the model
CREATE OR REPLACE MODEL
  `momo.churn_classifier`
OPTIONS (
  model_type = 'logistic_reg',
  label_cols = ['is_churned']
) AS
SELECT * FROM `momo.user_features`
WHERE split = 'TRAIN';
-- Step 2: Evaluate
SELECT * FROM ML.EVALUATE(
  MODEL `momo.churn_classifier`);
-- Step 3: Predict
SELECT * FROM ML.PREDICT(
  MODEL `momo.churn_classifier`,
  (SELECT * FROM `momo.users`));
```

Supported Model Types

Linear Regression

Revenue / price prediction

Logistic Regression

Churn, fraud, binary classify

K-Means Clustering

Customer segmentation

Matrix Factorization

Product recommendation

ARIMA+ Time Series

Demand & trend forecasting

Deep Neural Network

Complex classification tasks

XGBoost / Random Forest

Tabular data, structured ML

Import TF / PyTorch

Bring your own trained model

11.2 BigQuery as a Lakehouse — Open Formats & Integration

BigQuery goes beyond a data warehouse — it now operates as a full Lakehouse by supporting open table formats, modern transformation tools, and multi-cloud query.

Apache Iceberg Tables in BigQuery

Native Iceberg support since 2023. Query Iceberg tables on GCS with BigQuery SQL — ACID transactions, time-travel queries, partition evolution. Data stays in open format: no vendor lock-in. Interoperable with Spark and Flink.

dbt + BigQuery Integration

dbt transforms raw data inside BigQuery using SQL SELECT statements. Define models as .sql files, run tests, generate data lineage documentation. The most widely adopted ELT tool — free open-source core, 40,000+ companies.

Google Dataform

GCP-native transformation tool built into the BigQuery console. SQL workflows with version control (Git), scheduling, dependency management, and assertion-based data quality checks — zero extra infrastructure.

BigQuery Omni (Multi-cloud)

Query data stored in AWS S3 (us-east-1) or Azure ADLS without moving data between clouds. Dremel engine deployed inside AWS/Azure regions. One SQL interface for your entire multi-cloud data estate.

11.3 BigQuery Pricing Model

On-Demand

Pay per query

\$6.25 / TB scanned

- ✓ Free tier: 1 TB queries/month free
- ✓ Storage: \$0.02/GB/month (active data)
- ✓ Best for: irregular queries, startups, small teams
- ✓ Practical: 100 GB query $\approx$ \$0.63 USD

Most Popular

Capacity Pricing

Flat-rate compute slots

~\$1,700/slot/month

- ✓ Buy dedicated compute slots (parallelism units)
- ✓ Unlimited queries — no per-TB charge
- ✓ Best for: large teams with predictable query volume
- ✓ Break-even point: >400 TB/month

Enterprise

BigQuery Editions

Standard / Enterprise / Enterprise+

Slot-based + feature tier

- ✓ Standard: analytics, no CMEK, no BI Engine
- ✓ Enterprise: HA, CMEK, committed discounts
- ✓ Enterprise+: cross-region failover, advanced features
- ✓ Launched 2023 — replaces legacy flat-rate model

Since 2023

11.4 BigQuery vs Amazon Redshift vs Azure Synapse Analytics

Feature	BigQuery (GCP)	Redshift (AWS)	Synapse Analytics (Azure)
Architecture	Serverless, storage/compute split	Server-based (RA3), can separate	Serverless + dedicated pools
SQL Dialect	ANSI 2011, arrays, structs	PostgreSQL-based SQL	T-SQL (SQL Server dialect)
Scaling	Fully automatic, instant	Manual resize or elastic resize	Auto-pause; manual for dedicated
Pricing Model	Per TB scanned / slot reservation	Per node-hour (cluster size)	Per DWU (Data Warehouse Unit)
ML Integration	BigQuery ML — native SQL	SageMaker (external service)	Azure ML (external service)
Lakehouse Support	Iceberg, BigLake, Omni	Redshift Spectrum (query S3)	Lake Database + Delta Lake
Governance	Data Catalog, DLP, IAM, Audit Logs	Lake Formation, Glue Catalog	Purview, Synapse RBAC
Best For	GCP ecosystem, ad-hoc analytics	AWS ecosystem, PostgreSQL users	Azure / Microsoft stack users

CHAPTER 12

Summary & Q&A

Full journey recap, learning roadmap, and open discussion

12.1 Full Lecture Recap — The Data Engineering Journey

Ch.1	Foundations	Data Engineering role & lifecycle · Big Data 5Vs · Real-world data scale
Ch.2	On-Premise Era	Traditional DW · ETL model · Hadoop ecosystem · Data Lake origins · DL vs DW
Ch.3	Limitations	CAPEX constraints · Batch-only performance · Ops burden · Growth ceiling
Ch.4	Cloud Migration	Why cloud · On-prem vs cloud · ETL→ELT paradigm shift (expanded) · Batch vs stream
Ch.5	Cloud Ecosystem	Ingest→Store→Process→Analyze layers · AWS / Azure / GCP services · comparison
Ch.6	Use Cases — Vietnam	Tiki, Shopee, Grab, Zalo, MoMo, VPBank — real architectures and outcomes
Ch.7	Data Governance ★	Governance pillars · 6 quality dimensions · Pipeline validation · Cloud DLP & compliance
Ch.8	Current Trends	Lakehouse · dbt / Dataform · Streaming-first · AI/ML pipelines · Iceberg · Serverless
Ch.9-11	BigQuery Deep Dive	Dremel+Colossus architecture · SQL · Partition+Cluster · BQML · Lakehouse · Pricing

12.2 Suggested Learning Roadmap for Aspiring Data Engineers

Phase 1

Foundations (1–2 months)

- ▶ SQL mastery: joins, window functions, CTEs, subqueries
- ▶ Python basics: pandas, NumPy, file I/O, virtual environments
- ▶ Linux command line & Git version control fundamentals
- ▶ Basic statistics: mean, variance, distributions, correlation

Phase 2

Core Data Engineering (2–3 months)

- ▶ Apache Spark (PySpark) — batch & stream processing
- ▶ Apache Airflow — DAG-based pipeline orchestration
- ▶ dbt — SQL transformations, testing, and data modeling
- ▶ Cloud fundamentals: AWS / Azure / GCP associate-level certification

Phase 3

Cloud-Native & Specialization (3–4 months)

- ▶ BigQuery / Redshift / Synapse — hands-on end-to-end projects
- ▶ Streaming: Kafka or Pub/Sub + Dataflow / Flink pipelines
- ▶ Data Lakehouse: Apache Iceberg or Delta Lake in practice
- ▶ Data Governance: Great Expectations, dbt tests, data quality SLAs

Phase 4

Production & Advanced (ongoing)

- ▶ Build & deploy a full data pipeline end-to-end on real data
- ▶ MLOps: integrate ML models into production data pipelines
- ▶ Portfolio: GitHub projects + Kaggle + personal data engineering blog
- ▶ Target cert: GCP Professional Data Engineer or AWS Data Analytics Specialty



Questions & Discussion

- ✦ Data Engineering is the invisible infrastructure that makes data-driven organizations possible
- ✦ Cloud solves on-premise limitations: elastic scaling, managed services, and pay-as-you-go economics
- ✦ Data Governance is not optional — bad data quality destroys ML models, dashboards, and decisions
- ✦ BigQuery exemplifies modern serverless data warehousing: fast, scalable, SQL-native, ML-integrated